

MAP: Multi-user Personalization with Collaborative LLM-powered Agents

Christine P. Lee*

Department of Computer Sciences
University of Wisconsin-Madison
Madison, Wisconsin, USA
cplee5@cs.wisc.edu

Jihye Choi*

Department of Computer Sciences
University of Wisconsin-Madison
Madison, Wisconsin, USA
jihye@cs.wisc.edu

Bilge Mutlu

Department of Computer Sciences
University of Wisconsin-Madison
Madison, Wisconsin, USA
bilge@cs.wisc.edu

Abstract

The widespread adoption of Large Language Models (LLMs) and LLM-powered agents in multi-user settings underscores the need for reliable, usable methods to accommodate diverse preferences and resolve conflicting directives. Drawing on conflict resolution theory, we introduce a user-centered workflow for multi-user personalization comprising three stages: *Reflection*, *Analysis*, and *Feedback*. We then present MAP—a Multi-Agent system for multi-user Personalization—to operationalize this workflow. By delegating subtasks to specialized agents, MAP (1) retrieves and reflects on relevant user information, while enhancing reliability through agent-to-agent interactions, (2) provides detailed analysis for improved transparency and usability, and (3) integrates user feedback to iteratively refine results. Our user study findings ($n = 12$) highlight MAP’s effectiveness and usability for conflict resolution while emphasizing the importance of user involvement in resolution verification and failure management. This work highlights the potential of multi-agent systems to implement user-centered, multi-user personalization workflows and concludes by offering insights for personalization in multi-user contexts.

CCS Concepts

• **Human-centered computing** → HCI design and evaluation methods; • **Computing methodologies** → Natural language processing; • **Computer systems organization** → Robotics.

Keywords

personalization; human-robot interaction; large-language models

ACM Reference Format:

Christine P. Lee, Jihye Choi, and Bilge Mutlu. 2025. MAP: Multi-user Personalization with Collaborative LLM-powered Agents. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*, April 26–May 01, 2025, Yokohama, Japan. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3706599.3719853>

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
CHI EA '25, Yokohama, Japan

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1395-8/25/04
<https://doi.org/10.1145/3706599.3719853>

1 Introduction

As artificial intelligence (AI) continues to advance, its deployment in software, smart products, and robots is expanding into increasingly complex real-world scenarios [20, 21, 41]. These AI systems are becoming more autonomous in perceiving and interacting with their surroundings, exhibiting qualities such as reactivity, social engagement, and the ability to learn and adapt [8, 46]. In practice, these AI systems are often introduced into *multi-user environments*, such as smart home devices for families, automated scheduling for teams, and assistive robots in shared living spaces [24, 32, 40]. These contexts require AI systems to accommodate multiple users by integrating diverse individual preferences and instructions into a cohesive decision-making process. This highlights the need for AI to effectively manage complexity and adapt for seamless integration into such environments.

Imagine an AI-powered household robot receiving conflicting requests from different family members [24]—one user asks to keep the kitchen counter untouched due to recent caulking, while another unknowingly requests groceries to be placed on the same counter. In such situations, an ideal adaptive response from the robot would involve reconciling these conflicting instructions by recalling previously stated preferences, aligning its actions with the collective needs of all users. Additionally, the robot could generate explanations for its altered actions and accept updated instructions regarding the kitchen counter or groceries. Such adaptability can also enhance the long-term usability of AI systems by storing and retrieving user preferences when needed, reducing the need for users to repeatedly express their preferences or reestablish context.

For AI systems to effectively adapt and personalize in multi-user environments, they require clear frameworks that guide their personalization efforts. These frameworks should enable AI systems to engage in ongoing learning, accommodate diverse user requests, and adjust their operations to meet evolving needs and preferences. Moreover, they must consider the unique complexities introduced by multi-user dynamics. Addressing overlapping or conflicting preferences requires decision-making, prioritization, and, at times, the abandonment of certain requests. Successfully managing these conflicts and achieving resolutions that ensure a satisfactory and accepted user experience is a crucial challenge as AI systems become more integrated into multi-user settings.

In this work, we aim to develop guidelines and demonstrate our approach to providing personalized experiences with AI systems in multi-user environments. Our approach leverages recent advancements in large language models (LLMs), which have proven to be powerful tools for developing adaptive personalization solutions and aligning with human preferences through methods such as

reward function modeling [22, 50] and fine-tuning [5, 35]. Specifically, we utilize an orchestration of multiple LLM agents to collaborate on complex conflict resolution and preference adaptation tasks [12, 13]. Our research is guided by the central question: “How should LLM-powered AI systems be designed and built to provide adaptive personalization for multiple users?” To address this question, we introduce a user-centered workflow tailored for multi-user personalization, structured around the unique challenges of multi-user environments—particularly conflict resolution. Drawing from conflict resolution literature, we propose a three-stage framework: reflection, analysis, and feedback. We then present MAP (Multi-Agent for Multi-User Personalization), an implementation of our workflow using a collaborative orchestration of LLM-powered agents. We then assess the effectiveness of our designed workflow and MAP through a user study to understand user experiences and design requirements for AI systems deployed in multi-user settings.¹

2 Interaction Design of MAP

This section outlines the interaction design of MAP, focusing on the challenges and requirements of multi-user personalization. We first differentiate single-user and multi-user personalization to highlight the complexities of accommodating multiple users (Section 2.1). We then introduce real-world scenarios illustrating these challenges (Section 2.2). Finally, we present a structured workflow—*Reflection, Analysis, and Feedback*—to guide MAP’s approach to managing personalization and conflict resolution (Section 2.3).

2.1 Single-user vs Multi-user Personalization

Here we define our target task, highlighting the key differences between single-user and multi-user personalization. The tasks and interaction requirements for human-AI interactions differ significantly depending on whether the personalization task involves a single user or multiple users. In *single-user* interactions, the system focuses on adapting to the preferences and rules of an individual. The primary goal is to efficiently manage and personalize tasks to meet the unique needs of the individual without encountering external conflicts. In contrast, *multi-user* interactions introduce a more complex dynamic. The system must accommodate diverse and often conflicting preferences from multiple users. Requirements extend beyond basic personalization to include the identification and resolution of conflicts between users’ preferences and schedules. This creates a more complicated circumstances, where balancing competing needs and resolving conflicts become central tasks, leading to more sophisticated requirements for the AI system.

2.2 Everyday Scenarios

Throughout this paper, we focus on scenarios designed to simulate the multi-user personalization task, representing situations that commonly arise in daily life among multiple users. Each scenario involves interactions between an AI system and three hypothetical users, each with a weekly schedule and a set of preferences dictating how specific tasks should be handled. The objective in each scenario is for the AI system or users to generate daily plans that effectively accommodate the diverse schedules and preferences of

all users. Although these scenarios are not exhaustive or highly sophisticated, they aim to provide a meaningful and representative foundation for examining complex multi-user tasks. Extended scenario descriptions can be found in Appendix B.

- (1) *Scenario 1: Workplace Scheduling.* An AI system schedules meeting rooms for a consulting firm, balancing employee preferences (e.g., ambiance, lighting, seating) and resolving conflicts by prioritizing client consultations, team meetings, brainstorming, and other activities in order.
- (2) *Scenario 2: Assistive Care Robot.* An AI-powered robot in an assistive care facility manages residents’ preferences and schedules, such as providing interactive wake-up calls, mobility assistance, and custom deliveries. Conflicts are resolved by prioritizing requests alphabetically by residents’ names.
- (3) *Scenario 3: Smart-home Temperature Control.* A smart-home AI regulates communal area temperatures based on housemates’ schedules and preferences, such as warm settings for study or cooler ones for cost-saving. Conflicts are resolved through discussions among housemates.

2.3 Multi-user Personalization Workflow: Reflection, Analysis, and Feedback

Managing conflicts among users in multi-user personalization systems presents unique challenges, such as understanding the dynamic nature of user preferences, facilitating coordination in resolving conflicts, and providing transparency in decision-making processes. Addressing these complexities necessitates a principled approach that enables the systems to consistently provide conflict resolution strategies under evolving conditions. To this end, we draw on principles from the human conflict resolution theory [37, 42] to develop technical solutions to effectively mediate and resolve conflicts.

Specifically, Ross [36] introduces an *Action Evaluation* methodology that integrates goal setting, monitoring, and evaluation directly into the conflict resolution process. This methodology is structured around three iterative stages: reflection, analysis, and feedback. Adopting this structure, we propose a three-stage workflow for AI systems managing multi-user personalization:

- (1) *Reflection:* The system collects and aggregates personalization elements from various users, including preferences, interaction history, and schedules. This stage mirrors the participatory nature of Action Evaluation by involving all stakeholders in defining their needs and goals.
- (2) *Analysis:* The system synthesizes the collected data to construct a comprehensive plan that adheres to multiple users’ needs. When conflicts arise, the system generates resolutions that account for individual preferences and shared objectives. The final plan is presented to users, detailing the decision-making process, the rules considered, conflicts detected, and the proposed resolutions. This stage parallels the intervention planning in Action Evaluation, which incorporates mechanisms to address disagreements to achieve goals defined in the reflection stage.
- (3) *Feedback:* The system gathers user feedback on the generated plan. This feedback may include adjustments to existing settings, updates to user-defined preferences, or suggestions to

¹We provide an extended discussion on related literature regarding personalization with LLMs and LLM-based agents in Appendix A.

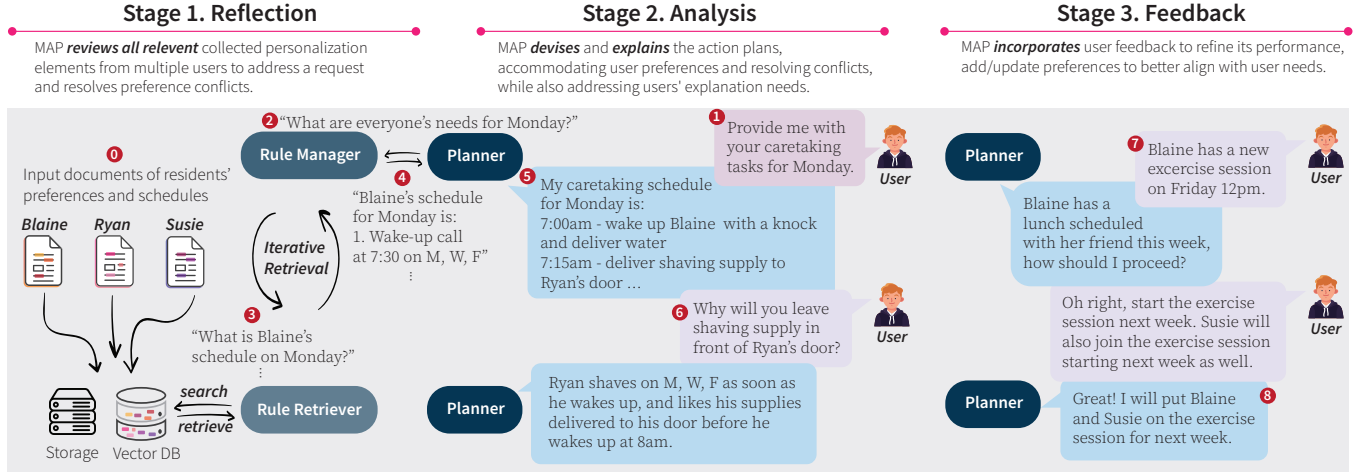


Figure 1: Our multi-agent system to support the proposed multi-user personalization workflow — The system orchestrates specialized agents through three stages of our user-centered workflow—Reflection, Analysis, and Feedback—to retrieve user data, reason about personalization tasks, resolve conflicts, and incorporate user feedback.

refine conflict resolution strategies. The system incorporates this feedback into its planning, for iterative improvement and alignment with user needs over time. This continuous refinement embodies the feedback stage of Action Evaluation, which revisits and refines the goals to ensure continued progress and alignment with participants' evolving needs.

By incorporating these stages, the proposed workflow ensures that AI systems managing multi-user personalization can mediate conflicts, adapt to dynamic user needs, and uphold transparency and satisfaction across diverse stakeholders. Building on Action Evaluation's principles of iterative goal setting and evaluation, our approach emphasizes the importance of transparency, adaptability, and user engagement in AI-mediated conflict resolution.

3 MAP: Multi-Agent LLMs for Multi-User Personalization

Recent advances in LLMs have led to the emergence of agents capable of handling complex tasks [7, 11, 15, 29, 44]. In contrast to a static, standalone LLM, an agent is a dynamic, intelligent entity that leverages an LLM's capabilities without task-specific training, by incorporating function-calling/tools or external knowledge bases. By coordinating multiple agents, each tasked with a specific sub-task, these systems work collaboratively to address more complex, nuanced end-tasks such as multi-user personalization. Here we propose MAP, a multi-agent LLM-based system that supports user-centered workflows introduced in Section 2.2: Reflection, Analysis, and Feedback. Refer to Appendix C for further details.²

At the core of MAP is an agent called the *Planner*, which serves as the highest-level interface between the user and the system. When given a user query such as "Provide me with your caretaking tasks for Monday" (step 1 in Figure 1), the system must (1) collect all relevant information to answer, and (2) reason about this information and generate the personalized response to the user. Rather than

assigning both (1) and (2) exclusively to the *Planner*, we offload (1) to specialized supporting agents (i.e., *Rule Retriever* and *Rule Manager*). This division frees the *Planner* to focus on (2)³.

Stage 0: Data Ingestion. Initially, users are asked to furnish their preferences in documents (depicted as step 0). These documents contain information pertinent to each user, encompassing individual requests, schedules, or rules. Subsequently, a pre-processing, or *ingestion phase* ensues, wherein documents are not just stored in a storage but also sharded into reasonable-size chunks and ingested into a suitable type of document store (e.g., vector database). This document store serves as a form of external storage where all users' personal information and preferences are stored, which is desired to be incorporated into the interactions of human users with the MAP system.

Stage 1: Reflection. To instantiate the reflection phase of the personalization workflow, the *Planner* agent is scheduled to collect all relevant user information before providing any personalized plans to the user prompts (step 2). *Rule Retriever* and *Rule Manager* agents effectively support this process. *Rule Retriever*, an LLM-powered agent with Retrieval Augmented Generation (RAG), retrieves a requested piece of information by directly accessing the document store from the data ingestion phase; it retrieves the top-*k* most *relevant* document-chunks to a given query. To further enhance retrieval quality, *Rule Retriever* is managed by another LLM-powered agent, called *Rule Manager*. *Rule Manager* breaks the *Planner*'s query into more granular, specifically executable sub-queries (e.g., "What is Blaine's schedule?") and iteratively sends each of them to the *Rule Retriever* (3) until all information is gathered (4).

³Such a (1) vs. (2) distinction draws on recent insights from the LLM uncertainty literature [1, 16, 27, 49], which identifies two key factors determining the reliability of LLM outputs: (1) the quality of demonstrations in the prompt, and (2) the LLM's internal reasoning capabilities. Note that (1) corresponds to the Reflection phase, while (2) corresponds to the Analysis and Feedback phases in our multi-user personalization workflow.

²The code repository for our work is available at <https://github.com/jihyechoi77/MAP>.

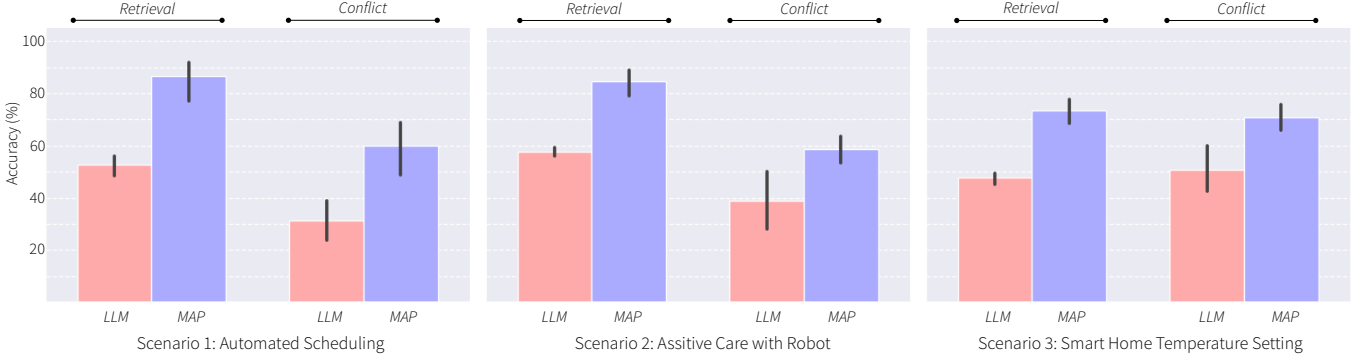


Figure 2: Monolithic LLM-based approach (“LLM”) vs. our multi-agent approach (“MAP”) across the three multi-user personalization scenarios—The evaluation measures each system’s success (in %) of 1) completely retrieving all 60 user preferences and schedules to generate personalized action plans (“Retrieval”), and 2) identifying and resolving all 12 present conflicts among the users (“Conflict”).

Stage 2: Analysis. After receiving the compiled information from the Rule Manager in Stage 1, the Planner generates personalized answers to the original query by the user (step 5). In doing so, it resolves any conflicting preferences among multiple users, and articulates the reasoning behind its responses; providing a summary of relevant rules referred to, upon request (6). Prompts used for the Planner can be found in Appendix D.

Stage 3: Feedback. The user offers feedback on the Planner’s response (step 7). The Planner maintains a conversation history, leveraging a large context window to keep track of previous queries and feedback. Future iterations could further improve personalization by ingesting new user inputs or additional context back into the document store (8).

3.1 Limitations of a Monolithic LLM Approach for Multi-User Personalization

Recent LLMs [2–4] are well known for their long-enough context window, advanced reasoning capabilities, and handling nuanced language—qualities that seem well-suited for complex personalization tasks. One might therefore wonder if a basic chat agent, implemented as a monolithic LLM with a conversation history, could sufficiently support the proposed user-centric workflow in Section 2.2. However, we emphasize the advantages of a modular design, in which the overarching task is decomposed into subtasks that are distributed among multiple agents. Such a composition-based approach not only enhances overall system reliability (as we demonstrate in the quantitative evaluation below) but also provides flexibility to adapt to evolving changes in the personalization workflow. As the complexity of personalization grows—with more users, more intricate rules, or additional user feedback—an architecture composed of specialized agents can more readily incorporate new features (e.g., failure-handling functionality as discussed in Section 4.2) without requiring extensive rewrites to the entire system.

Furthermore, in Figure 2, we compare the effectiveness of a monolithic LLM (LLM, red bars) and our multi-agent framework (MAP, blue bars) across three personalization scenarios described

in Section 2.2. For the monolithic approach, we instantiate a single GPT-4-based chat agent. Our evaluation focuses on two main criteria: (1) *Retrieval*, reflecting how comprehensively the system accounts for user rules; and (2) *Conflict Resolution*, measuring how well it identifies and resolves conflicts among multiple users—a core challenge in multi-user personalization. In our evaluation, we assessed the accuracy of both the single-agent LLM and MAP. Each scenario involved 60 rules governing the preferences and schedules of three users, leading to 12 conflicts due to overlapping preferences and schedules. Each model was tested three times across three scenarios. Since each test generated five daily action plans for weekdays, this resulted in 15 instances per scenario and 45 instances per condition, totaling 90 instances for validation. We then compared each generated output against a reference solution formulated by the researcher who designed the scenarios. To evaluate accuracy, we measured errors where the models failed to correctly align with the intended schedules or preferences. The results in Figure 2 indicate that MAP significantly outperforms the monolithic LLM across all scenarios. This performance gap arises in part because the monolithic LLM’s ability to resolve conflicts is hampered by its limited retrieval performance. For instance, with a limited retrieval rate (~50%) of user preferences and schedules, it often fails to generate responses that fully account for all user rules, leading to inadequate conflict resolution (~30% at the lowest). Typical errors include overlooking scheduling overlaps (e.g., multiple users requesting the same meeting room at 12:00 pm on Friday) and ignoring conflicting requests (e.g., one user’s temperature ceiling of 70°C versus another’s preference for 80°C). By contrast, the multi-agent architecture ensures more comprehensive retrieval and thus more robust conflict resolution.

4 User Study of MAP

4.1 Study Design

We conducted a user study to evaluate the effectiveness of the multi-user personalization workflow and MAP. The study used a within-subjects design with scenarios outlined in Section 2.2 as the within-subjects variable. Participants were introduced to MAP’s workflow

stages (reflection, analysis, and feedback) and given examples to familiarize themselves with its interaction process. Each participant was assigned two of three scenarios in a counterbalanced order and tasked with creating weekly plans considering varied preferences and scheduling requirements.

Participants interacted with MAP through its web interface, navigating the workflow stages and evaluating the system's proposed plans and conflict resolution strategies. Qualitative data was gathered via semi-structured interviews conducted in person, with sessions recorded on Zoom [53]. Each session lasted approximately one hour, and participants were compensated \$15 per hour. Session recordings were transcribed using Otter.ai [33] and manually reviewed. Qualitative analysis followed thematic analysis [9, 30], with open coding performed by the first author to identify key concepts, which were then clustered into themes. The research team iteratively refined the themes through discussion and consensus.

4.1.1 Participant Demographics. 12 participants (P1–P12) were recruited through university mailing lists for our user study. Eligibility criteria included being in the United States, fluent in English, and at least 18 years old. Participants ranged in age from 20 to 56 years ($M = 30.6$, $SD = 10.8$), with 58.4% identifying as female and 41.6% as male. The racial composition was 83.4% White, 8.3% Asian, and 8.3% preferring not to respond. Findings are reported using the notation P_i , where i represents the participant ID.

4.2 Findings

Our analysis provided insights into participants' perceptions of the proposed workflow and their interaction experience with MAP. Below, we present the key themes identified in our findings.

4.2.1 Conflict Resolution as an AI-Suitable Task in Multi-user Settings. Ten participants (P1–P5, P7, P8, P10–P12) described conflict detection and resolution as a suitable task for AI, as it is an unfavorable and undesired responsibility for humans. They explained that conflicts in multi-user settings are inevitable, as differences in preferences and overlapping schedules frequently arise, and users are often unaware of each other's needs and potential conflicts.

Participants further elaborated that resolving such conflicts among multiple people is mentally and emotionally draining. The cognitive load of tracking various preferences and schedules, coupled with the challenge of predicting potential conflicts, makes the task complicated and time-consuming. As P4 expressed: *"I can't possibly keep track of what everyone in the house prefers, nor predict every conflict when planning things. It's overwhelming. Having the system not only identify these conflicts but also resolve them for me is a huge relief—it saves so much time and prevents unnecessary arguments or assumptions."* Consequently, participants found that analyzing such information and generating mitigating plans is an ideal task for the AI system. Moreover, they envisioned that the role of the system in resolving conflicts could be adaptively used even in high-stakes tasks, where it could detect conflicts and suggest solutions, while still allowing humans to confirm or adjust the final resolution.

4.2.2 Reflection Stage Enhancing Usability. Participants highlighted that the system's ability to save user preferences and automatically retrieve information to generate plans significantly improved usability. This improvement in usability stemmed from two key aspects:

- (1) the automatic creation of plans for complex specifications and
- (2) the ease of not having to prompt manually.

Eleven participants (P1–P10, P12) reported that the system's management and generation of schedules and preferences reduced their effort and time, easing the mental load of integrating diverse inputs. P3 explained: *"I can't hold all the preferences in my head, so it was really helpful that the system organized [preferences]. All I had to do was confirm and think of new fun rules to add. When I did it manually, I struggled just to ensure basic rules were met. The system really took the load off me."* While manual management was feasible, participants noted that accommodating diverse preferences was particularly challenging without automation.

Four participants (P3, P7, P11, P2) further emphasized that the system's ability to retrieve information without requiring repeated user input streamlined the interaction experience. This feature allowed users to build on existing information rather than repeatedly entering extensive preferences and schedules. Participants envisioned such functionality enhancing usability in real-world settings, enabling users to expand upon prior lists with minimal effort.

4.2.3 Analysis and Feedback Stage Supporting Transparency and Adaptability. Eight participants (P2, P3, P4, P6–P9, P12) emphasized the importance of the analysis and feedback stages, particularly working together to achieve effective personalization. They highlighted that the analysis stage was important for system transparency and output verification, while the feedback stage enabled adaptability to address the needs identified during analysis.

The analysis stage provided accessible and understandable information through MAP's interface and natural language capabilities, helping participants clarify the system's decision-making logic and data sources (e.g., other users' preferences and schedules) based on their needs. Participants also valued the analysis stage for verifying the system's outputs and logic. Four participants (P3, P6, P7, P12) specifically used these explanations to better understand the decision-making process by comparing outputs against the provided list of rules and resolving uncertainties. They emphasized that such explanations were key to building trust with the system. For instance, P6 remarked: *"As I asked more questions and received additional details, it clarified my doubts and blind spots. I felt more confident, thinking, 'Okay, I can trust this system and try more complex tasks, but not too extreme ones, since I can always ask questions and provide feedback if needed.'"* By fostering transparency and verification of planning and resolution outcomes, the analysis stage played a crucial role in enhancing user acceptance.

Building on the insights gained from the analysis stage, participants applied this information to adapt MAP's behavior to their individual or group needs and preferences. Seven participants (P1, P2, P3, P8–P11) noted that the feedback stage was key to ensuring adaptability, allowing the system to respond to evolving needs and group norms. They highlighted the dynamic nature of preferences and schedules, with P9 explaining: *"The initial preferences are never set in stone. Things come up, and if a system designed to help me can't change with me, it's not very helpful."* Some participants (P2, P3, P8–P10) further emphasized the role of feedback in aligning the system with group-specific norms and priorities. As P10 observed: *"These rules reflect what we, as a group, want and value, so the system needs to follow and blend into our preferences,"* highlighting the

feedback mechanism’s importance in addressing both individual and collective needs.

4.2.4 Support Needed for Retrieval and Conflict Resolution Failures. During system engagement, participants (P1, P3, P4, P6, P9, P11, P12) identified issues where the system failed to retrieve relevant rules, misdetected conflicts, or generated inconsistent resolutions. For example, one participant noted, P1: “Sometimes it followed my preferences at the expense of my roommate’s, and other times it did the opposite. This unpredictability makes it hard to work with or plan around.” While participants found the automated planning and conflict resolution features useful, these inconsistencies raised concerns about overreliance on the system and skepticism in the reliability of the resolution capabilities.

To address these issues, participants expressed a need for robust explanations and verification mechanisms. They wanted clarity on which rules were considered, which were missed, and how conflicts were resolved. One participant highlighted, P9: “Given that it’s complicated and likely to make choices [in what rules to choose], I want to just check what choices were made, to make sure it’s the best choice for all of us.” Another suggested the system provide comparisons between processed rules and the full rule list, flagging omissions. P4: “It would be helpful if the system could flag where it’s unsure about certain preferences, so I know to pay closer attention to those areas.” These features were seen as critical for navigating system shortcomings and enabling better-informed decision-making.

5 Discussion and Future Work

In this work, we focus on enabling AI systems to provide personalization in multi-user settings, where unique and complex dynamics often lead to conflicting preferences and needs. To address these challenges, we propose a workflow inspired by conflict resolution theory, which guides AI systems through three critical stages: reflection, analysis, and feedback. Our findings demonstrate that this three-stage workflow effectively facilitates transparent and adaptive personalization while efficiently managing the complexity of balancing multiple users’ needs. Each stage plays a crucial role in cooperatively navigating and addressing multi-user demands, showing promise for extension to more complex scenarios.

Our work highlights the importance of embedding a user-centric workflow into the design of multi-agent systems. By starting with the workflow requirements (*i.e.*, how humans resolve conflicts in the real-world) and then constructing agents around those requirements, we show how AI systems can better serve users—particularly in critical domains like multi-user personalization. However, as our system revealed technical limitations, such as retrieval failures, the workflow can be adapted to incorporate user control where needed while leveraging LLM capabilities effectively. For instance, to mitigate retrieval failures, the system can generate multiple retrieval attempts and plans, then incorporate user input to rank the options or provide feedback for refinement.

Our study also revealed the need for greater user involvement and verification tools within the workflow, as well as mechanisms to address conflict resolution failures. Future work could explore enhancing the MAP framework with additional agents that further engage users, for example through critic or validator roles [7, 29].

Such agents could monitor and correct the Planner’s outputs, ensure responses adhere to specific user constraints, or invoke user feedback and external function calls to adjust rules, preferences, or weighting schemes [14]. In addition, programming language methods such as formal verification and repair can be integrated into LLMs (*e.g.*, [23]). Formal verification techniques, such as model checking, can impose deterministic boundaries to counteract the probabilistic nature of LLMs. These boundaries, representing user-defined preferences and rules, can ensure that LLM outputs strictly adhere to specified constraints. Such methods could address the inconsistent conflict resolutions observed in MAP by providing structured guidelines for coordination. Future work can explore applying formal methods, like program synthesis, verification, and repair, to enhance LLM reliability and alignment with user needs.

Another avenue for investigation is extending these ideas to larger-scale personalization scenarios—incorporating more users, more complex preference structures, and additional types of task conflicts. By broadening the scope, researchers can explore both the scalability of our proposed agentic approach and the reliability of its conflict resolution processes.

Acknowledgments

We thank the reviewers for their helpful comments. We also thank the participants in our study. This work was supported by the National Science Foundation awards 1925043 and 2152163. Jihye Choi is partially supported by DARPA under agreement number 885000, NSF CCF-FMiTF-1836978 and ONR N00014-21-1-2492.

References

- [1] Yasin Abbasi-Yadkori, Ilja Kuzborskij, András György, and Csaba Szepesvari. 2024. To Believe or Not to Believe Your LLM: Iterative Prompting for Estimating Epistemic Uncertainty. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=k6iyUfwd19>
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [3] Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Anja Hauth, Katie Millican, David Silver, Slav Petrov, Melvin Johnson, Ioannis Antonoglou, Julian Schrittwieser, Amelia Glaese, Jilin Chen, Emily Pitler, Timothy Lillicrap, Angeliki Lazaridou, Orhan Firat, et al. 2023. Gemini: A Family of Highly Capable Multimodal Models. *arXiv:2312.11805* [cs.CL]
- [4] Anthropic. 2024. The Claude 3 Model Family: Opus, Sonnet, Haiku. (2024). https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf
- [5] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862* (2022).
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [7] Jihye Choi, Nils Palumbo, Prasad Chalasani, Matthew M. Engelhard, Somesh Jha, Anivarya Kumar, and David Page. 2024. MALADE: Orchestration of LLM-powered Agents with Retrieval Augmented Generation for Pharmacovigilance. In *Machine Learning for Healthcare 2024*.
- [8] Nazli Cila. 2022. Designing human-agent collaborations: Commitment, responsiveness, and support. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–18.
- [9] Victoria Clarke and Virginia Braun. 2014. Thematic analysis. In *Encyclopedia of critical psychology*. Springer, 1947–1952.
- [10] Jeff Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.

- [11] Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. 2023. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325* (2023).
- [12] Chen Gao, Xiaochong Lan, Nian Li, Yuan Yuan, Jingtao Ding, Zhilun Zhou, Fengli Xu, and Yong Li. 2024. Large language models empowered agent-based modeling and simulation: A survey and perspectives. *Humanities and Social Sciences Communications* 11, 1 (2024), 1–24.
- [13] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680* (2024).
- [14] Jiuzhou Han, Wray Buntine, and Ehsan Shareghi. 2024. Towards Uncertainty-Aware Language Agent. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 6662–6685. <https://doi.org/10.18653/v1/2024.findings-acl.398>
- [15] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VtmBAGCN7o>
- [16] Bairu Hou, Yujian Liu, Kaizhi Qian, Jacob Andreas, Shiyu Chang, and Yang Zhang. [n. d.]. Decomposing Uncertainty for Large Language Models through Input Clarification Ensembling. In *Forty-first International Conference on Machine Learning*.
- [17] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for NLP. In *International conference on machine learning*. PMLR, 2790–2799.
- [18] Qiushi Huang, Shuai Fu, Xubo Liu, Wenwu Wang, Tom Ko, Yu Zhang, and Lilian Tang. 2023. Learning Retrieval Augmentation for Personalized Dialogue Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2523–2540.
- [19] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).
- [20] Callie Y Kim, Christine P Lee, and Bilge Mutlu. 2024. Understanding Large-Language Model (LLM)-powered Human-Robot Interaction. *arXiv preprint arXiv:2401.03217* (2024).
- [21] Tzu-Sheng Kuo, Hong Shen, Jisoo Geum, Nev Jones, Jason I Hong, Haiyi Zhu, and Kenneth Holstein. 2023. Understanding frontline workers' and unhoused individuals' perspectives on ai used in homeless services. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–17.
- [22] Minae Kwon, Sang Michael Xie, Kalesha Bullard, and Dorsa Sadigh. 2023. Reward design with language models. *arXiv preprint arXiv:2303.00001* (2023).
- [23] Christine Lee, David Porfrio, Xinyu Jessica Wang, Kevin Zhao, and Bilge Mutlu. 2025. VeriPlan: Integrating Formal Verification and LLMs into End-User Planning. *arXiv preprint arXiv:2502.17898* (2025).
- [24] Christine P Lee, Pragathi Praveena, and Bilge Mutlu. 2024. Rex: Designing user-centered repair and explanations to address robot failures. In *Proceedings of the 2024 ACM designing interactive systems conference*. 2911–2925.
- [25] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.
- [26] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. 2023. Holistic Evaluation of Language Models. *Transactions on Machine Learning Research* (2023). <https://openreview.net/forum?id=iO4LZibEqW> Featured Certification, Expert Certification.
- [27] Chen Ling, Xujiang Zhao, Xuchao Zhang, Wei Cheng, Yanchi Liu, Yiyu Sun, Mika Oishi, Takao Osaki, Katsushi Matsuda, Jie Ji, et al. 2024. Uncertainty Quantification for In-Context Learning of Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 3357–3370.
- [28] Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. 2023. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. *arXiv preprint arXiv:2308.08747* (2023).
- [29] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2024. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems* 36 (2024).
- [30] Nora McDonald, Sarita Schoenebeck, and Andrea Forte. 2019. Reliability and Inter-rater Reliability in Qualitative Research: Norms and Guidelines for CSCW and HCI Practice. *Proceedings of the ACM on Human-Computer Interaction* 3 (11 2019), 1–23. <https://doi.org/10.1145/3359174>
- [31] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems* 26 (2013).
- [32] Bilge Mutlu and Jodi Forlizzi. 2008. Robots in organizations: the role of workflow, social, and environmental factors in human-robot interaction. In *Proceedings of the 3rd ACM/IEEE international conference on Human robot interaction*. 287–294.
- [33] Otter.ai. 2023. Otter.ai: Voice Meeting Notes. <https://otter.ai>. "Accessed = 03-01-2024".
- [34] Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. 1532–1543.
- [35] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems* 36 (2024).
- [36] Marc Howard Ross. 2001. Action evaluation in the theory and practice of conflict resolution. *Peace and Conflict Studies* 8, 1 (2001), 1–15.
- [37] Rothman. 1997. Action evaluation and conflict resolution training: Theory, method and case study. *International Negotiation* 2, 3 (1997), 451–470.
- [38] Jingqing Ruan, Yihong Chen, Bin Zhang, Zhiwei Xu, Tianpeng Bao, Hangyu Mao, Ziyue Li, Xingyu Zeng, Rui Zhao, et al. 2023. Tptu: Task planning and tool usage of large language model-based ai agents. In *NeurIPS 2023 Foundation Models for Decision Making Workshop*.
- [39] Alireza Salemi, Sheshera Mysore, Michael Bendersky, and Hamed Zamani. 2024. LaMP: When Large Language Models Meet Personalization. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7370–7392.
- [40] Isabella Seeber, Eva Bittner, Robert O Briggs, Triparna De Vreede, Gert-Jan De Vreede, Aaron Elkins, Ronald Maier, Alexander B Merz, Sarah Oeste-Reiß, Nils Randrup, et al. 2020. Machines as teammates: A research agenda on AI in team collaboration. *Information & management* 57, 2 (2020), 103174.
- [41] Dakota Sullivan, Nathan Thomas White, Andrew Schoen, and Bilge Mutlu. 2024. Making Informed Decisions: Supporting Cobot Integration Considering Business and Worker Preferences. *arXiv preprint arXiv:2401.05587* (2024).
- [42] GS Thyagaraju, SM Joshi, Umakant P Kulkarni, SK NarasimhaMurthy, and Anil R Yardi. 2008. Conflict resolution in multiuser context-aware environments. In *2008 International Conference on Computational Intelligence for Modelling Control & Automation*. IEEE, 332–338.
- [43] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [44] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155* (2023).
- [45] xAI. [n. d.]. grok-1. ([n. d.]). <https://github.com/xai-org/grok-1>
- [46] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. 2023. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864* (2023).
- [47] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. *arXiv:2309.07597* [cs.CL]
- [48] Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. 2021. An Explanation of In-context Learning as Implicit Bayesian Inference. In *International Conference on Learning Representations*.
- [49] Zhangyue Yin, Qiushi Sun, Qipeng Guo, Zhiyuan Zeng, Xiaonan Li, Junqi Dai, Qinyuan Cheng, Xuanjing Huang, and Xipeng Qiu. 2024. Reasoning in Flux: Enhancing Large Language Models Reasoning through Uncertainty-aware Adaptive Guidance. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 2401–2416. <https://doi.org/10.18653/v1/2024.acl-long.131>
- [50] Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montse Gonzalez Arenas, Hao-Tien Lewis Chiang, Tom Erez, Leonard Hasenclever, Jan Humplik, et al. 2023. Language to rewards for robotic skill synthesis. *arXiv preprint arXiv:2306.08647* (2023).
- [51] Abhiman Neelakanteswara Shreyas Chaudhari Hamed Zamani. 2024. RAGs to Style: Personalizing LLMs with Style Embeddings. In *The 1st Workshop on Personalization of Generative AI Systems*. 119.
- [52] Yizhe Zhang, Siqi Sun, Michel Galley, Yen-Chun Chen, Chris Brockett, Xiang Gao, Jianfeng Gao, Jingjing Liu, and William B Dolan. 2020. DIALOGPT: Large-Scale Generative Pre-training for Conversational Response Generation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. 270–278.
- [53] Zoom. 2023. Video Conferencing Platform. "Accessed = 09-29-2023".

A Related Work: Personalization and Adaptation in the Era of LLMs

In the era dominated by LLMs, encompassing commercial LLMs (e.g., GPT-4 [2], Gemini families [3], and Claude [4]) and “open source” LLMs (e.g., LLaMA [43], Grok [45], and Mistral [19]), we have witnessed the remarkable capabilities of these models in generating human-like text and understanding contextual cues. Nonetheless, just having an LLM generate answers based on its pre-training has a clear limitation: although the most-capable LLMs have been trained on large swaths of the internet, they would not have seen documents generated past a specific training cutoff date, and certainly could not have seen personal/enterprise documents, documents behind paywalls, data in databases, and so on. To overcome such limitation, there are two common approaches for incorporating domain-specific information into LLMs for their personalized usage: fine-tuning and Retrieval Augmented Generation (RAG).

Adaptation with Fine-tuning. Fine-tuning involves adapting the pre-trained LLM to a specific task or user data by continuing the training process, thereby enabling the models to learn the nuances and requirements of the target domain [17, 39, 52]. However, the fine-tuning approach requires substantial computational resources and expertise, as well as full white-box access to the model’s internals, which is not a realistic assumption since LLM model weights are often protected by the model provider’s intellectual property laws. Moreover, it risks catastrophic forgetting, where the model loses its ability to perform tasks it was previously good at, and the severity of forgetting increases as the model scale increases [28]. Other than adaptation to target task, Reinforcement Learning from Human Feedback (RLHF) can help align LLMs with user preferences [5]. RLHF involves the additional steps of training a Reward Model with preference data, which consists of prompts and multiple candidate outputs ranked by human evaluators. Then it uses the reward model to fine-tune the LLM through reinforcement learning to align the LLM’s behavior with human preferences and values defined via reward modeling. Yet, the performance of RLHF largely depends on reward modeling and the quality and quantity of human feedback, which can be challenging to obtain, and it shares the computational expense and complexity of the fine-tuning approach. This makes fine-tuning and RLHF-based approaches less accessible to individual users who may want to leverage state-of-the-art LLMs for personal use off-the-shelf.

Adaptation without Fine-tuning. On the other hand, RAG provides more flexibility to adapt to changing data without retraining the entire model, by only requiring black-box access to the model (i.e., via API calls). It allows the model to leverage knowledge sources (e.g., specific documents on personal information or instructions, or an external database) not present in the original training data of the deployed LLM. It retrieves relevant information from the external knowledge and uses it to guide the response generation by augmenting the user input prompt [25]. This method is not only computationally efficient and versatile, compared to fine-tuning or RLHF approaches that necessitate additional training, but is also accessible to individuals who wish to adaptively use any black-box LLMs for personal use cases. More importantly, it is possible with RAG to verify the validity of the generated answer by including a source citation.

There have been a few recent attempts to explore the use of RAG in the context of personalized dialogue generation, such as persona profiles and distinctive authorial nuances [18, 51]. However, the focus of personalization in our work is more complicated; it rather lies in handling conflicting multiple user requests and supporting decision-making adapted to individual preferences and instructions.

Beyond an LLM to LLM-powered agent. Other than RAG, another emerging paradigm in LLM applications is *tool-use*, where LLM is instructed to produce structured outputs (e.g., in JSON) that downstream code can interpret to perform tasks such as web searches, API queries, and computations [38]. By integrating RAG and tool-use, LLMs can function as agents – an intelligent entity that can interact with each other by exchanging messages, collaborate with each other and provide feedback, just like a team of humans. By harnessing the collective intelligence of LLMs, multi-agent systems can tackle more complex tasks with greater reliability than a single LLM agent even with RAG or tool-use [7, 15]. To the best of our knowledge, our work is the first to extend multi-agent collaboration to multi-user adaptive personalization, moving beyond the monolithic LLM-based approaches commonly found in the prior works. Crucially, rather than retrofitting an existing system, we design a user-centered personalization workflow first and explicitly embed this workflow into the development of our multi-agent orchestration framework.

To the best of our knowledge, our work is the first to extend the potential of multi-agent collaboration into the domain of multi-user adaptive personalization, in contrast to the monolithic agent-based approaches found in most prior works on RAG-based personalization. More importantly, we prioritize designing a usable personalization workflow from the users’ perspectives, then integrate it explicitly into the development of our multi-agent orchestration framework.

B Extended Descriptions of Multi-user Personalization Scenarios

Scenario 1: Automated Scheduling System for Workplaces. A consulting firm utilizes an AI system to manage the scheduling of shared meeting spaces. Employees exhibit diverse preferences for these spaces—some prioritize a room for its professional ambiance and cooler climate, whereas others favor a different space due to its natural lighting and suitability for collaborative efforts. Their preferences extend to features such as whiteboards, ambient music, chairs arranged in a circular layout, and the provision of materials like sticky notes and flip charts. Each employee follows a distinct schedule involving client consultations, team meetings, brainstorming sessions, and other engagements. Conflicts arise when multiple individuals concurrently request the same room, necessitating the introduction of a priority system. This system assigns the highest priority to client consultations, followed by team meetings, brainstorming sessions, and then all other activities, to streamline the allocation of meeting spaces.

Scenario 2: Assistive Care with Robot. An assistive care facility employs an AI-powered social robot to support the care of senior residents. Each resident possesses distinct preferences and schedules, necessitating tailored approaches to ensure their comfort and well-being. For instance, one resident prefers interactive wake-up calls and wants assistance for mobility in morning walks, communal breakfasts, and participation in various activities throughout the week. Conversely, another resident prioritizes independence and privacy, preferring minimal interaction with the robot and requesting that all deliveries be left at her door. Regarding schedules, each resident follows a personalized routine that includes unique wake-up times on different days, weekly appointments with doctors and physical therapists, participation in external activities such as yoga, art classes, and book clubs, as well as visits from family members. In instances where there are conflicts between schedules and preferences, the proposed resolution involves prioritizing requests based on the alphabetical order of the residents' first names.

Scenario 3: Smart-home Temperature Setting with Housemates. An AI-powered smart home device is tasked with regulating the temperature in communal areas of a residence shared by three housemates. Each housemate has personal temperature preferences and varying schedules for when these preferences should be applied. For instance, one housemate desires a warmer temperature in the study area during her hours of study or remote work. Another housemate insists on keeping the temperature at or below a certain threshold to minimize electricity costs. Meanwhile, a third housemate opts for cooler temperatures when hosting parties and warmer settings during his workout sessions in the home gym. Furthermore, each housemate adheres to individual schedules that include work, rest, exercise, sleep, and social events throughout the week. In cases where there are conflicts between their schedules and temperature preferences, the housemates have established a consensus to resolve such issues through discussion among those involved.

C Implementation Details of MAP

C.1 Preliminaries: Design Primitives of LLM-based Agents

While today's LLMs exhibit impressive capabilities, they remain constrained by technical and practical limitations such as brittleness, non-determinism, limited context window, inference costs, and latency [26, 48]. In other words, one cannot simply give high-level instructions to an LLM and expect it to accomplish a task, with these limitations becoming more pronounced as the complexity of the target task or application increases. Consequently, to best harness the capability of an LLM as a component of a complicated task, it is necessary to decompose the task into sub-tasks and manage multiple LLM conversations, each equipped with its own set of specifically-defined instructions, state, and data sources.

Agent. A natural and convenient abstraction for managing such complexity is the notion of an agent who is instructed to be responsible for a specific aspect of the task. An agent is essentially an "intelligent" entity that can respond to, and transform messages. Typically, an agent encapsulates an LLM (e.g., an interface to GPT-4 in our implementation) and may also be equipped with so-called *tools* (also known as functions or plugins) and *memory* such as conversation history or a vector database (described below). Much like a team of humans, agents interact by exchanging messages. And importantly, an *orchestration mechanism* is needed to manage the flow of messages between agents, to ensure that progress is made towards the completion of the task, and to handle the inevitable cases where an agent deviates from instructions. In this work we adopt this multi-agent paradigm, where agents are first-class citizens, acting as message transformers, and communicate by exchanging messages.

Tools. As elucidated above, an LLM is essentially a text transformer; i.e., in response to some input text (known as a *prompt*), it produces a response. Free-form text responses are ideal when we want to generate a description, answer, or summary for human consumption, or even a question for another agent to answer. However, in some cases, we need the responses to trigger external *actions*, such as a database query, web search, API call, or code execution. Then, we instruct the LLM response to be *structured*, typically in JSON format, with various pre-specified fields, such as code, an SQL query, parameters of an API call, and so on. These structured responses have come to be known as tools, and the LLM is said to use a tool when it produces a structured response corresponding to a specific tool. To elicit a tool response from an LLM, it needs to be instructed on the expected tool format and the conditions under which it should use the tool. Such tool-use capability allows LLM-based agents to leverage external tools and resources to accomplish tasks enhancing their functional capabilities, and operate more effectively in diverse and dynamic environments.

Memory. In contrast to a solitary LLM, an agent maintains memory, which encompasses accumulated message history, detailing all queries and responses involving the agent, or external databases, facilitating in-context learning [6] to retain and retrieve information over prolonged periods. This capability enhances contextual coherence and fosters learning from interactions, crucial for navigating complex tasks efficiently.

As mentioned above, in constructing a system with multiple agents to tackle intricate tasks, an orchestration mechanism is critical to managing message flow between agents, ensuring task progression, and handling deviations from instructions. In our implementation, we build upon the Langroid library, which offers a simple yet versatile orchestration mechanism for agents, seamlessly managing user interaction, tool utilization, and sub-task delegation.

C.2 Implementation Details

Our framework first asks the users to write down their adaptation requests or rules into documents and then preprocesses them into a database (*data ingestion*). Referring to the personalized data, the three core agents in MAP collaborate with each other to support the three phases in MAP framework: reflection, articulation, and feedback. The reflection stage is facilitated by the *Rule Manager* and *Rule Retriever* agents; Rule Retriever serves as the lowest-level agent that directly extracts structured information from the database, while the medium-level agent, Rule Manager ensures that Rule Retriever extracts all users' relevant information. At the highest level, the *Planner* agent initiates interaction with the user and incorporates the hierarchical orchestration with the Rule Manager and Rule Retriever. It handles the articulation and feedback stages seamlessly; it generates personalized responses with the guidance of the Rule Manager, addresses conflicts that may arise in satisfying all users' preferences, and provides explanations for its responses upon request. Below we describe the implementation details of the three agents, and tasks, tools, and resources assigned to each of them. For an illustrative description of the overall process of MAP, refer to Figure 1.

Data Ingestion. At the onset of the interaction, users are asked to furnish their preferences in documents. These documents contain information pertinent to each user, encompassing individual requests, schedules, or rules. Subsequently, a pre-processing, or *ingestion phase* ensues, wherein documents are sharded into reasonable-size chunks. Each document chunk is mapped to a vector embedding via an embedding model (e.g., BAAI/bge-large-en-v1.5 [47] in our implementation). This process capitalizes on the inherent ability of vector embeddings to encapsulate the meaning of sentences or paragraphs [10, 31, 34]. Each vector embedding is then ingested into a vector database, along with a pointer to the chunk contents as metadata. This database is a form of external storage where all users' personal information and preferences are stored, which is desired to be incorporated into the interactions of human users with the AI system.

Rule Manager. The Rule Manager is tasked with extracting structured information from documents and presenting it in a specific nested format. The JSON structure is explicitly defined for all users, where the highest-level fields represent users (e.g., Blaine, Ryan, and Susie in Figure 1). Under each user field, the Rule Manager defines subfields to maximize information retrieval from the document. Rule Manager generates questions corresponding to each field in the nested format and passes them, one by one, to the Rule Retriever who has direct access to documents (more specifically, the vector database constructed from the ingestion phase). If the Rule Retriever cannot provide an answer to a question, the Rule Manager rephrases the question and retries a few more times until an answer is obtained. Once all fields in the structured format are filled, the Rule Manager returns the retrieved information to the Planner using a tool message.

Rule Retriever. The Rule Retriever has direct access to documents and vector database, and is responsible for answering questions generated by the Rule Manager. Given a question, this agent retrieves the top- k most *relevant* document-chunks from the vector database. It employs a combination of two notions of relevance: (a) *lexical relevance*, so-called keyword search, which is based on word-overlap between the query and the document-chunk, and (b) *semantic relevance*, which is a more flexible notion of relevance based on similarity of meaning between the query and the document-chunk. To determine semantic relevance, the question is mapped to a vector embedding using the same embedding model as in the ingestion phase, and the top- k nearest-neighbors of this vector (based on cosine similarity) are found from the vector database, and their corresponding document-chunks are retrieved. The original question from Rule Manager is augmented with the retrieved document-chunks, and given the augmented question, Rule Retriever provides an answer to Rule Manager.

D Prompt Used for Planner

Here is the excerpt from the Planner's system prompt for each scenario.

Scenario 1: Workplace Scheduling.

You are a helpful automated scheduler in my office.
 You arrange workers' meetings to rooms, and tell the receptionist if anything else needs to be prepared.
 In a meeting room, workers often meet for updates, brainstorming sessions, and even little workplace birthday parties.
 Additionally, the room is used to consult clients.
 You cater to the individual needs and preferences of each worker,
 ensuring a more productive and comfortable meeting environment for everyone involved.

I will give you delivery, and you should give me (the User) personalized answers throughout the conversation.
 You MUST retrieve BOTH schedules and preferences of EVERY users, and
 MAKE SURE that NONE of your answers conflicts with ANY of the rules and preferences that you figured out
 based on the document.

Workers have different preferences for setting up the room depending on who they are meeting with and
 what they find comfortable.
 Workers prefer the Sun room, as this is the biggest meeting room,
 but if that is booked for a prior meeting, assign workers to use the Apple room for meetings.
 IF CONFLICT OCCURS between workers needing the meeting rooms, PRIORITIZE giving the Sun room then the Apple room
 in the order of client consultation, team meetings, brainstorming sessions, and then everything else.
 If the Sun and Apple rooms are unavailable, suggest another time.

Once you present your answer, the I may ask for the summary of relevant rules that you referred to.
Then you should provide the summary of relevant rules.

Scenario 2: Assistive Care Robot.

You are a helpful assistive caretaking robot.

I will give you delivery, and you should give me (the User) personalized answers throughout the conversation.
You MUST retrieve ALL schedules and preferences of EVERY users, and
MAKE SURE that NONE of your answers conflicts with ANY of the rules and preferences that you figured out
based on the document.

When assisting residents, if ANY resident has overlapping conflicts from their preferences
and schedules with another resident, serve them in ALPHABETICAL ORDER of the resident's name.
Once you present your answer, the I may ask for the summary of relevant rules that you referred to.
Then you should provide the summary of relevant rules.

Scenario 3: Smart-home Temperature Control.

You are a helpful smart home agent to set temperatures in specific spaces automatically.
Roommates all have different schedules and preferences for temperature settings.
Roommates have all agreed that 70 degrees would be the normal temperature setting for the house.

I will give you delivery, and you should give me (the User) personalized answers throughout the conversation.
You MUST retrieve BOTH schedules and preferences of EVERY users, and
MAKE SURE that NONE of your answers conflicts with ANY of the rules and preferences that you figured out
based on the document.

Once you present your answer, the I may ask for the summary of relevant rules that you referred to.
Then you should provide the summary of relevant rules.